1. Which keyword is used to declare a user-defined class blueprint in Python?

(A) struct
(B) class
(C) object
(D) blueprint

2. What is an instance of a class called in Python?

(A) Module
(B) Object
(C) Function
(D) Variable

3. Which special method serves as the constructor / initializer for newly instantiated objects?

(A) __new__
(B) __init__
(C) __create__
(D) __construct__

4. What does the first parameter 'self' conventionally represent in an instance method?

(A) The class blueprint itself
(B) The specific instance object invoking the method
(C) The global module namespace
(D) The Python Virtual Machine

5. Is the exact identifier name 'self' a reserved keyword in Python syntax?

(A) Yes, it is a strict keyword like 'class'
(B) No, it is purely a strong community naming convention
(C) Yes, in Python 3.10+
(D) Only in abstract classes

6. How are attributes and methods accessed on an object in Python?

(A) Arrow operator ->
(B) Dot operator .
(C) Colon operator ::
(D) Bracket notation [] only

7. Which built-in function checks whether an object possesses a specific attribute by string name?

(A) hasattr(obj, 'attr')
(B) getattr(obj, 'attr')
(C) checkattr(obj, 'attr')
(D) isin(obj, 'attr')

8. Which built-in function dynamically retrieves the value of a named attribute from an object?

(A) hasattr()
(B) getattr(obj, 'attr'[, default])
(C) setattr()
(D) readattr()

9. Which built-in function sets the value of a named attribute on an object dynamically?

(A) putattr()
(B) setattr(obj, 'attr', value)
(C) assign()
(D) update()

10. Which built-in function deletes an attribute from an object?

(A) delattr(obj, 'attr')
(B) removeattr()
(C) drop()
(D) del.attr()

11. What is a Class Variable in Python?

(A) A variable defined inside an instance method
(B) A variable defined directly inside the class body, shared by all instances
(C) A variable defined in the global module
(D) A private variable

12. What is an Instance Variable in Python?

(A) A variable tied to a specific object instance (typically bound via self.var)
(B) A variable shared across all instances
(C) A constant defined in bytecode
(D) A local loop variable

13. What attribute on an instance holds its writable attribute namespace as a dictionary?

(A) __slots__
(B) __dict__
(C) __class__
(D) __namespace__

14. Which OOP principle allows a derived class to inherit fields and methods from a base class?

(A) Encapsulation
(B) Polymorphism
(C) Inheritance
(D) Abstraction

15. What built-in function returns a proxy object delegating method calls to a parent or sibling class?

(A) parent()
(B) base()
(C) super()
(D) ancestor()

16. Which built-in function checks if an object is an instance of a specified class or subclass?

(A) isinstance(obj, Class)
(B) issubclass(obj, Class)
(C) typeof(obj, Class)
(D) is_a(obj, Class)

17. Which built-in function checks if a class is derived from another class?

(A) isinstance()
(B) issubclass(Derived, Base)
(C) is_parent()
(D) has_base()

18. What is Method Overriding in Python?

(A) Defining multiple methods with the same name but different arguments
(B) A subclass providing a specific implementation of a method already defined in its superclass
(C) Hiding methods behind private flags
(D) Deleting methods dynamically

19. What is Polymorphism in object-oriented programming?

(A) The ability to run Python on multiple operating systems
(B) The ability of different classes to respond to the same method interface in their own distinct ways
(C) Creating multiple instances of a single class
(D) Inheriting from 10 classes

20. What Python programming philosophy emphasizes object behavior over explicit type inheritance ('If it walks like a duck and quacks like a duck...')?

(A) Static Polymorphism
(B) Strict Nominal Typing
(C) Duck Typing
(D) Structural Encapsulation

21. How does Python signal that an attribute or method is intended as 'protected' (internal convention)?

(A) With a leading single underscore: _variable
(B) With a leading double underscore: __variable
(C) With the 'protected' keyword
(D) With an '@protected' decorator

22. How does Python enforce private attribute encapsulation and prevent name collisions in subclasses?

(A) With a leading double underscore: __variable (triggers Name Mangling)
(B) With the 'private' keyword
(C) By throwing a compiler error
(D) By encrypting the attribute

23. To what internal format does Python mangle a private attribute __data inside class Student?

(A) _data
(B) _Student__data
(C) __Student_data
(D) private.Student.data

24. Can name-mangled private attributes (_ClassName__var) still be accessed from outside in Python?

(A) No, hardware prohibits access
(B) Yes, Python offers encapsulation by convention rather than strict access restriction
(C) Only if decrypted with SSL
(D) Only in debug mode

25. Which decorator is used to define managed attributes with getter methods?

(A) @property
(B) @getter
(C) @attribute
(D) @accessor

26. How is a setter method declared for a property named 'price'?

(A) @price.setter
(B) @property.setter
(C) @set.price
(D) @price.set

27. Does Python support traditional compile-time function overloading with multiple signatures?

(A) Yes, natively using duplicate def statements
(B) No, defining a function twice silently overwrites the previous definition with the newest one
(C) Yes, using the @overload decorator alone at runtime
(D) Only for integer parameters

28. How can multiple constructor behaviors or signatures be emulated in Python?

(A) Using default parameter values (*args, **kwargs) or @classmethod factory methods
(B) By writing multiple __init__ methods in the same class
(C) By creating multiple class blueprints
(D) Python cannot emulate multiple constructors

29. What are special double-underscore methods in Python commonly called?

(A) Hidden methods
(B) Magic methods / Dunder methods
(C) Secret routines
(D) Core macros

30. Which dunder method is invoked by the built-in str() function and print() for informal human-readable representation?

(A) __repr__
(B) __str__
(C) __print__
(D) __format__

31. Which dunder method is invoked by repr() and the interactive REPL for unambiguous, formal developer representation?

(A) __str__
(B) __repr__
(C) __eval__
(D) __dump__

32. Which dunder method overloads the addition operator (+)?

(A) __plus__
(B) __add__
(C) __sum__
(D) __concat__

33. Which dunder method overloads the subtraction operator (-)?

(A) __sub__
(B) __minus__
(C) __diff__
(D) __reduce__

34. Which dunder method overloads the multiplication operator (*)?

(A) __mul__
(B) __times__
(C) __product__
(D) __star__

35. Which dunder method overloads the true division operator (/)?

(A) __div__
(B) __truediv__
(C) __floordiv__
(D) __divide__

36. Which dunder method overloads the floor division operator (//)?

(A) __truediv__
(B) __floordiv__
(C) __mod__
(D) __intdiv__

37. Which dunder method overloads the equality comparison operator (==)?

(A) __eq__
(B) __equals__
(C) __same__
(D) __match__

38. Which dunder method overloads the less-than comparison operator (<)?

(A) __lt__
(B) __gt__
(C) __le__
(D) __less__

39. Which dunder method overloads the len() built-in function on custom objects?

(A) __count__
(B) __size__
(C) __len__
(D) __dimension__

40. Which dunder method allows custom instances to be invoked like functions: obj()?

(A) __call__
(B) __invoke__
(C) __run__
(D) __exec__

41. What is the output of:\nclass A:\n count = 0\na1 = A(); a2 = A()\na1.count = 5\nprint(a1.count, a2.count, A.count)

(A) 5 0 0
(B) 5 5 5
(C) 5 0 5
(D) Error

42. Why did a1.count = 5 in Q41 NOT modify A.count or a2.count?

(A) Assignment to a1.count creates an instance attribute on a1, shadowing the class attribute
(B) Class attributes are immutable
(C) a2 has higher priority
(D) CPython creates a copy of A

43. What occurs when modifying a mutable class attribute: A.items.append(1)?

(A) Modifies items only for class A
(B) Modifies the shared list for all instances accessing A.items
(C) Raises an AttributeError
(D) Creates an alias

44. What decorator defines a method receiving the class object (cls) as its first argument rather than the instance (self)?

(A) @staticmethod
(B) @classmethod
(C) @instancemethod
(D) @property

45. What decorator defines a method that receives neither self nor cls, behaving as an isolated plain function scoped inside the class?

(A) @classmethod
(B) @staticmethod
(C) @isolated
(D) @plain

46. What is the output of:\nclass B:\n def __repr__(self): return 'B_repr'\n def __str__(self): return 'B_str'\nb = B()\nprint(str(b), repr(b))

(A) B_str B_repr
(B) B_repr B_str
(C) B_str B_str
(D) B_repr B_repr

47. If a class defines __repr__() but omits __str__(), what does print(obj) display?

(A) Raises an AttributeError
(B) Falls back to __repr__() output automatically
(C) Displays <Object at 0x...>
(D) Displays None

48. What is the output of:\nclass Vector:\n def __init__(self, x, y): self.x, self.y = x, y\n def __add__(self, o): return Vector(self.x + o.x, self.y + o.y)\nv = Vector(1, 2) + Vector(3, 4)\nprint(v.x, v.y)

(A) 4 6
(B) 1 2 3 4
(C) TypeError
(D) Vector(4, 6)

49. What happens when an expression 5 + custom_obj is evaluated if custom_obj's class does not implement __radd__?

(A) TypeError: unsupported operand type(s) for +: 'int' and 'Custom'
(B) Automatically swaps operands and calls __add__
(C) Returns 5
(D) SyntaxError

50. Which dunder method handles right-hand operand arithmetic when the left operand does not support the operation: 10 + obj?

(A) __ladd__
(B) __radd__
(C) __right_add__
(D) __reverse_add__

51. What dunder method enables index subscript access: val = obj[key]?

(A) __index__
(B) __getitem__
(C) __setitem__
(D) __lookup__

52. What dunder method enables index subscript assignment: obj[key] = val?

(A) __getitem__
(B) __setitem__
(C) __putitem__
(D) __assign__

53. What dunder method enables the 'in' membership test: item in custom_obj?

(A) __has__
(B) __contains__
(C) __in__
(D) __membership__

54. What is the output of:\nclass Parent:\n def show(self): return 'Parent'\nclass Child(Parent):\n def show(self): return super().show() + ' -> Child'\nprint(Child().show())

(A) Parent -> Child
(B) Child -> Parent
(C) Parent
(D) Child

55. What algorithm does Python use to determine the Method Resolution Order (MRO) in multiple inheritance hierarchies?

(A) Depth-First Search (DFS)
(B) Breadth-First Search (BFS)
(C) C3 Linearization Algorithm
(D) Dijkstra's Algorithm

56. What property or method displays the MRO tuple of a class C?

(A) C.__mro__ or C.mro()
(B) C.__hierarchy__
(C) __order__
(D) C.get_parents()

57. In the classic Diamond Inheritance problem (D inherits from B and C, both inheriting from A), what is D's standard MRO order?

(A) [D, B, C, A, object]
(B) [D, B, A, C, object]
(C) [D, A, B, C, object]
(D) [D, C, B, A, object]

58. What exception is raised when defining a multiple inheritance structure whose inheritance graph cannot be resolved into a consistent linear order?

(A) InheritanceError
(B) TypeError: Cannot create a consistent method resolution order (MRO)
(C) ResolutionError
(D) SyntaxError

59. What is the purpose of the __slots__ declaration inside a class?

(A) Restricts allowed attributes and replaces instance __dict__ with a fixed-size array of descriptor pointers to save memory
(B) Enables multithreading locks
(C) Creates database slots
(D) Defines slot machine random variables

60. What happens if code attempts to assign an undeclared attribute to an instance of a class that defines __slots__ = ('x', 'y')?

(A) Attribute is added dynamically
(B) AttributeError: 'Point' object has no attribute 'z'
(C) TypeError
(D) Silent failure

61. How much memory reduction does __slots__ typically achieve for millions of lightweight objects?

(A) ~5% to 10%
(B) ~40% to 60% memory savings by eliminating the per-instance dictionary
(C) Zero savings
(D) 100%

62. What module in the Python standard library provides Abstract Base Classes (ABCs)?

(A) abstract
(B) abc
(C) interfaces
(D) contracts

63. Which decorator from the abc module marks a method that MUST be implemented by any concrete subclass?

(A) @abstractmethod
(B) @virtual
(C) @must_implement
(D) @interface

64. What happens when attempting to instantiate an Abstract Base Class that contains unimplemented @abstractmethod methods?

(A) Object is created with None stubs
(B) TypeError: Can't instantiate abstract class with abstract methods
(C) Warning is logged
(D) Instantiation succeeds

65. What is the output of:\nclass Account:\n def __init__(self, bal): self._b = bal\n @property\n def balance(self): return self._b\na = Account(100)\nprint(a.balance)

(A) 100
(B) <property object>
(C) AttributeError
(D) TypeError

66. What occurs if code executes a.balance = 200 for the Account class in Q65 when no setter is defined?

(A) Sets balance to 200
(B) AttributeError: property 'balance' of 'Account' object has no setter
(C) TypeError
(D) Creates a new variable

67. What is the output of:\nclass Greeter:\n def __call__(self, name): return f'Hello, {name}'\ng = Greeter()\nprint(g('Aarav'))

(A) 'Hello, Aarav'
(B) TypeError: 'Greeter' object is not callable
(C) 'Hello, name'
(D) None

68. What built-in function tests whether an object implements the __call__ protocol and can be invoked like a function?

(A) is_function()
(B) callable(obj)
(C) can_call(obj)
(D) invokable(obj)

69. What is the output of:\nclass Sample:\n def __del__(self): print('Cleaned')\ns = Sample()\ndel s

(A) Prints 'Cleaned' (when reference count drops to zero and __del__ destructor executes)
(B) No output
(C) SyntaxError
(D) Sample object remains in memory

70. Why is relying heavily on __del__ (destructors) for resource cleanup discouraged in enterprise Python?

(A) Destructor execution timing is non-deterministic and cyclic references can delay or prevent execution
(B) Python deletes objects immediately
(C) __del__ is deprecated
(D) It crashes the interpreter

71. What language construct is preferred over __del__ for deterministic resource management (closing files, releasing locks)?

(A) Context Managers (with statement implementing __enter__ and __exit__)
(B) Global cleanup threads
(C) Manual free() calls
(D) While loops

72. What does the dunder method __new__(cls, *args, **kwargs) do?

(A) Initializes existing attributes
(B) Allocates and returns the physical new instance object before __init__ is called
(C) Deallocates memory
(D) Compiles bytecode

73. What is the correct return type of __init__ in Python?

(A) The newly created object
(B) None (Must return None; returning anything else raises a TypeError)
(C) Integer memory address
(D) Boolean True

74. What occurs if __init__ explicitly returns a value: return 42?

(A) TypeError: __init__() should return None, not 'int'
(B) Returns 42 to caller
(C) Silently ignored
(D) Warning

75. What is the output of:\nclass Counter:\n _instances = 0\n def __init__(self): Counter._instances += 1\n_ = [Counter() for _ in range(5)]\nprint(Counter._instances)

(A) 5
(B) 1
(C) 0
(D) TypeError

76. What is a Descriptor in Python?

(A) A docstring formatted as Markdown
(B) An object attribute with 'binding behavior' whose attribute access is overridden by __get__, __set__, or __delete__ methods
(C) A JSON configuration file
(D) A memory pointer

77. Which methods constitute the Python Descriptor Protocol?

(A) __get__, __set__, __delete__
(B) __read__, __write__, __erase__
(C) __init__, __new__, __del__
(D) __enter__, __exit__

78. What is the difference between a Data Descriptor and a Non-Data Descriptor?

(A) Data descriptors define both __get__ and __set__ (or __delete__); non-data descriptors define only __get__
(B) Data descriptors store only integers
(C) Non-data descriptors have no methods
(D) There is no difference

79. In attribute lookup resolution, which has higher priority: an instance's __dict__ or a Data Descriptor on its class?

(A) Instance __dict__ has higher priority
(B) Data Descriptor has higher priority (overrides instance dictionary entries!)
(C) Whichever was created first
(D) Random priority

80. What is a Metaclass in Python?

(A) A class that inherits from multiple bases
(B) The class of a class; a blueprint that defines how classes themselves are constructed and instantiated
(C) A C++ extension module
(D) A deprecated Python 2 feature

81. What is the default metaclass of all standard classes in Python 3?

(A) object
(B) type
(C) Class
(D) Meta

82. What is the relationship between 'type' and 'object' in Python's type hierarchy?

(A) type is an instance of type, and inherits from object; object is an instance of type
(B) object inherits from type
(C) They are identical singletons
(D) They have no relationship

83. How is a custom metaclass applied to a class definition?

(A) class MyClass(metaclass=MyMeta):
(B) class MyClass: __meta__ = MyMeta
(C) @metaclass(MyMeta) class MyClass:
(D) class MyClass extends MyMeta:

84. What hook method in Python 3.6+ allows parent classes to customize subclass creation without writing a full metaclass?

(A) __init_subclass__(cls, **kwargs)
(B) __new_subclass__
(C) __subclass_created__
(D) __on_inherit__

85. What does the C3 Linearization algorithm guarantee when calculating MRO?

(A) Strict alphabetical order of classes
(B) Local Precedence Order preservation and Monotonicity
(C) Fastest CPU instruction cache hit rate
(D) Smallest memory footprint

86. In cooperative multiple inheritance with super(), what does calling super().method() actually invoke?

(A) Always the direct syntactic parent written in the class definition
(B) The next class in the runtime MRO chain of the invoking instance
(C) The base object class
(D) The oldest ancestor

87. What is Duck Typing formalized by in modern Python (PEP 544)?

(A) Abstract Base Classes
(B) typing.Protocol (Structural Subtyping / Static Duck Typing)
(C) Enums
(D) Dataclasses

88. What does the @dataclass decorator (introduced in Python 3.7 via PEP 557) generate automatically?

(A) __init__, __repr__, __eq__, and optional comparison/hash methods based on type annotations
(B) SQL database tables
(C) Web API serializers
(D) C language headers

89. What parameter to @dataclass(frozen=True) enforces immutability?

(A) frozen=True makes instances read-only by raising FrozenInstanceError on attribute assignment
(B) immutable=True
(C) const=True
(D) readonly=True

90. What dunder method is invoked when an attribute is NOT found in the instance __dict__ or class tree?

(A) __getattribute__
(B) __getattr__
(C) __missing__
(D) __lookup_failed__

91. What is the critical difference between __getattr__ and __getattribute__?

(A) __getattribute__ is called unconditionally for EVERY attribute access; __getattr__ is called ONLY as a fallback when the attribute is absent
(B) They are identical aliases
(C) __getattr__ is for methods; __getattribute__ is for variables
(D) __getattr__ is deprecated

92. Why must implementations of __getattribute__ call super().__getattribute__(name) rather than self.__dict__[name]?

(A) To prevent infinite recursive loops calling __getattribute__ indefinitely
(B) To speed up lookup by 10x
(C) To check permissions
(D) Because __dict__ is private

93. What does the dunder method __missing__(self, key) do in a custom dictionary subclass?

(A) Removes missing keys
(B) Invoked by dict.__getitem__ when a requested key is absent from the dictionary
(C) Raises a KeyError
(D) Logs an error to disk

94. What is a Mixin Class in Python OOP design?

(A) A complete standalone application
(B) A small, specialized class providing reusable methods to multiple unrelated classes through multiple inheritance, not intended for standalone instantiation
(C) A class with both static and dynamic methods
(D) A class that converts data to JSON

95. What design pattern does Python's @property implement under the hood?

(A) Observer Pattern
(B) Descriptor Pattern
(C) Singleton Pattern
(D) Factory Pattern

96. What is the Singleton Design Pattern in Python?

(A) A pattern ensuring a class has only one instance and provides a global access point to it
(B) A class with only one method
(C) A function with one argument
(D) A tuple with one element

97. How is the Singleton Pattern commonly implemented in Python?

(A) By overriding the __new__ method to cache and return a single class-level instance
(B) By declaring the class private
(C) By running Python in single-threaded mode
(D) Using global variables only

98. What does the weakref module provide in Python OOP?

(A) Weak references that allow referencing an object without incrementing its reference count, preventing cyclic memory retention
(B) Low-security password hashing
(C) Slow pointer lookups
(D) Deprecated classes

99. What happens when an object with only weak references pointing to it has all strong references deleted?

(A) Object remains alive until program termination
(B) Object is immediately deallocated and weak reference returns None
(C) Raises MemoryError
(D) Weak reference throws an exception

100. What does __subclasses__() method on a class return?

(A) List of all direct subclasses currently loaded in memory
(B) Number of instances created
(C) List of parent classes
(D) MRO tuple

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the Object-Oriented Programming Paradigm in Python. Detail Classes, Instance Objects, the __init__ Initializer, and the 'self' Reference Parameter with Memory Models.

Q2. Contrast Class Attributes versus Instance Attributes in Python. Detail attribute shadowing, mutable class attribute bugs, and namespace lookup order.

Q3. Explain Methods in Python: Instance Methods, Class Methods (@classmethod), and Static Methods (@staticmethod). Compare signatures, use cases, and factory patterns.

Q4. Explain Dynamic Attribute Management in Python: hasattr(), getattr(), setattr(), delattr(), and the __getattr__ / __setattr__ magic fallback methods.

Q5. Explain Data Hiding and Encapsulation in Python: Public, Protected (_var), and Private (__var) attributes with Name Mangling and the @property Decorator.

Q6. Explain Single and Multilevel Inheritance in Python. Detail super() proxy initialization, method resolution, and the Liskov Substitution Principle (LSP).

Q7. Explain Multiple Inheritance, the Diamond Problem, and the C3 Linearization Algorithm (MRO) in Python with comprehensive structural traces.

Q8. Explain Polymorphism and Duck Typing in Python. Contrast Nominal Subtyping (isinstance) with Structural Subtyping (typing.Protocol).

Q9. Explain Operator Overloading in Python. Detail Arithmetic (__add__, __mul__), Relational (__eq__, __lt__), and String Representation (__str__, __repr__) magic methods.

Q10. Explain the Container Protocol and Sequence Emulation in Python: __len__, __getitem__, __setitem__, __delitem__, __contains__, and __iter__.

Q11. Explain the __slots__ Optimization in Python. Analyze how eliminating instance __dict__ reduces memory footprint and prevents dynamic attribute creation.

Q12. Explain Abstract Base Classes (ABCs) in Python. Detail the abc module, @abstractmethod decorator, interface enforcement, and virtual subclass registration.

Q13. Implement a Banking Domain Model in Python using OOP: BankAccount, SavingsAccount (with interest), and CheckingAccount (with overdraft limits).

Q14. Implement the Singleton Design Pattern in Python using both the __new__ override technique and a custom Metaclass. Analyze thread safety considerations.

Q15. Implement a Custom Matrix Class in Python with complete Operator Overloading (+, -, *, ==, str, repr, transpose) and dimension validation.

Q16. Explain the Descriptor Protocol in Python (__get__, __set__, __delete__). Implement a TypeChecked and BoundedNumeric descriptor for model validation.

Q17. Explain Mixin Classes and Cooperative Multiple Inheritance in Python. Implement a JSONSerializable and LoggingMixin system.

Q18. Implement a State Machine Pattern in Python using OOP: OrderState, PendingState, PaidState, ShippedState, and DeliveredState.

Q19. Explain the @dataclass Decorator (PEP 557) in Python. Contrast dataclasses with standard classes, namedtuples, and pydantic models.

Q20. Implement a Custom Iterator and Iterable Class in Python: FibonacciIterable supporting the Iterator Protocol (__iter__ and __next__).

Q21. Implement a Dependency Injection Container in Python using Classes and Reflection for service registration and resolution.

Q22. Implement the Observer Design Pattern in Python using Classes: EventDispatcher, Subject, and Concrete Observers.

Q23. Implement a Custom Context Manager in Python using OOP: DatabaseConnectionManager implementing __enter__ and __exit__ with exception handling.

Q24. Implement an Object Relational Mapping (ORM) Style Active Record Entity in Python using Metaclasses and Field descriptors.

Q25. Explain the Strategy Design Pattern in Python using Classes: PaymentProcessor with CreditCardStrategy, PayPalStrategy, and CryptoStrategy.

Q26. Implement a Custom Exception Hierarchy in Python for an E-Commerce Platform with base and derived domain exceptions.

Q27. Implement the Factory Method and Abstract Factory Design Patterns in Python using Classes for Cross-Platform UI Rendering.

Q28. Explain Metaclasses in Python in depth. Implement a ModelValidatorMeta metaclass that validates that all subclass attributes have docstrings.

Q29. Implement a Thread-Safe In-Memory Key-Value Store with TTL (Time-To-Live) Eviction in Python using Classes and threading.Lock.

Q30. Implement the Command Design Pattern in Python with Undo/Redo Capability using Classes and Command Stacks.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[B]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[B]	**Q81:**	[B]
Q2:	[B]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[B]	**Q82:**	[A]
Q3:	[B]	**Q23:**	[B]	**Q43:**	[B]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[B]	**Q24:**	[B]	**Q44:**	[B]	**Q64:**	[B]	**Q84:**	[A]
Q5:	[B]	**Q25:**	[A]	**Q45:**	[B]	**Q65:**	[A]	**Q85:**	[B]
Q6:	[B]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[B]	**Q86:**	[B]
Q7:	[A]	**Q27:**	[B]	**Q47:**	[B]	**Q67:**	[A]	**Q87:**	[B]
Q8:	[B]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[B]	**Q88:**	[A]
Q9:	[B]	**Q29:**	[B]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[B]	**Q50:**	[B]	**Q70:**	[A]	**Q90:**	[B]
Q11:	[B]	**Q31:**	[B]	**Q51:**	[B]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[B]	**Q52:**	[B]	**Q72:**	[B]	**Q92:**	[A]
Q13:	[B]	**Q33:**	[A]	**Q53:**	[B]	**Q73:**	[B]	**Q93:**	[B]
Q14:	[C]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[B]
Q15:	[C]	**Q35:**	[B]	**Q55:**	[C]	**Q75:**	[A]	**Q95:**	[B]
Q16:	[A]	**Q36:**	[B]	**Q56:**	[A]	**Q76:**	[B]	**Q96:**	[A]
Q17:	[B]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[B]	**Q38:**	[A]	**Q58:**	[B]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[B]	**Q39:**	[C]	**Q59:**	[A]	**Q79:**	[B]	**Q99:**	[B]
Q20:	[C]	**Q40:**	[A]	**Q60:**	[B]	**Q80:**	[B]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS Program in Python: Detail Classes, Instance Objects, the init Initializer, and the 'self' R

4. Complexity Analysis:
- Instantiation Time Complexity: O(1) attribute assignments.
- Space Complexity: O(K) where K is number of instance attributes in __dict__.

Question 2: Contrast Class Attributes versus Instance Attributes in Python. Detail attribute shadowing, mutable class attribute bugs, and namespac

Solution Statement:

- Lookup Time: O(1) hash table lookup across instance and class dictionaries.
- Space Complexity: O(1) attribute storage.

Solution to Question 3:

>>> # Explain Methods in Python: Instance Methods, Class Methods (@classmethod), and Static Methods (@staticmethod), Compare signatu

```
d1 = DateParser(2026, 10, 26)
d2 = DateParser.from_string("2024-02-29")

print(f"Date 1 (Standard Init): {d1.format_iso()}")
print(f"Date 2 (Factory String): {d2.format_iso()}")
print(f"Is 2024 a leap year? {DateParser.is_leap_year(2024)}")
print(f"Is 2026 a leap year? {DateParser.is_leap_year(2026)}")

3. Complexity Analysis:
- Method Call Overhead: O(1) descriptor binding (__get__ converts functions to bound methods).
- Space Complexity: O(1) memory per instantiated object.
```

>> Solution to Question 4: Explain Dynamic Attribute Management in Python: hasattr(), getattr(), setattr(), delattr(), and the __getattr__ / __setattr__ magic fallbac

```
# Dynamic attribute access via dot notation
print(f"Database Host: {config.database_host}")
print(f"Port: {config.port}")
print(f"Fallback Key: {config.missing_secret_key} (Dynamically handled!)")

# Programmatic reflection via getattr / setattr
setattr(config, "max_connections", 100)
print(f"hasattr max_connections: {hasattr(config, 'max_connections')}")
print(f"getattr max_connections: {getattr(config, 'max_connections')}")

4. Complexity Analysis:
- hasattr / getattr: O(1) average hash table lookup.
- Space Complexity: O(K) storage inside internal dictionary.
```

>> Solution to Question 5: Explain Data Hiding and Encapsulation in Python: Public, Protected (_var), and Private (__var) attributes with Name Mangling and the

```
    self.__balance = new_balance

    @balance.deleter
    def balance(self):
    raise PermissionError("Account balances cannot be deleted.")

    # Demonstration
    acct = BankAccount("Aarav", 5000.0)
    print(f"Owner: {acct.owner} | Balance: ${acct.balance:,.2f}")

    # Managed attribute setter in action
    acct.balance = 7500.0
    print(f"Updated Balance: ${acct.balance:,.2f}")

    # Validation enforcement
    try:
    acct.balance = -100.0
    except ValueError as err:
    print(f"Validation Caught: {err}")

    # Inspect Name Mangling
    print(f"Internal __dict__ Keys: {list(acct.__dict__.keys())}")
    print(f"Accessing mangled attribute directly: {acct._BankAccount__balance}")

    5. Complexity Analysis:
    - Property Access Overhead: O(1) descriptor __get__ / __set__ call.
    - Space Complexity: O(1) auxiliary frame memory.
```

>>Solution Statement Question 6: Explain Single and Multilevel Inheritance in Python. Detail super() proxy initialization, method resolution, and the Liskov Substitution Pri

```
def get_info(self) -> str:
return f"[{self.manufacturer}] VIN: {self.vin}"

class MotorVehicle(Vehicle):
"""Intermediate class in multilevel hierarchy."""
def __init__(self, vin: str, manufacturer: str, fuel_type: str):
super().__init__(vin, manufacturer)
self.fuel_type = fuel_type

def get_info(self) -> str:
return f"{super().get_info()} | Fuel: {self.fuel_type}"

class ElectricCar(MotorVehicle):
"""Leaf derived class."""
def __init__(self, vin: str, manufacturer: str, battery_kwh: float):
super().__init__(vin, manufacturer, fuel_type="Electric")
self.battery_kwh = battery_kwh

def get_info(self) -> str:
return f"{super().get_info()} | Battery: {self.battery_kwh} kWh"

# Verification
ev = ElectricCar("5YJSA1E2", "Tesla", 85.0)
print("Multilevel Inheritance Chain Info:")
print(ev.get_info())

print(f"isinstance(ev, Vehicle): {isinstance(ev, Vehicle)}")
print(f"issubclass(ElectricCar, Vehicle): {issubclass(ElectricCar, Vehicle)}")
print(f"ElectricCar MRO: {[c.__name__ for c in ElectricCar.mro()]}")

5. Complexity Analysis:
- Method Dispatch Time: O(1) cached MRO pointer lookup.
- Space Complexity: O(1) shared class methods.
```

>> Solution to Question 57: Explain Multiple Inheritance, the Diamond Problem, and the C3 Linearization Algorithm (MRO) in Python with comprehensive structural

1. Children precede parents: Subclasses always appear before their base classes.
2. Parent order preservation: Base classes appear in the exact order listed in the class header: class D(B, C) -> B precedes C.
3. Monotonicity: If Class X precedes Class Y in any parent's MRO, Class X will precede Class Y in all subclass MROs.

3. Python Code Demonstration:

```
class A:
def ping(self):
print("Ping from A")

class B(A):
def ping(self):
print("Ping from B")
super().ping()

class C(A):
def ping(self):
print("Ping from C")
super().ping()

class D(B, C):
def ping(self):
print("Ping from D")
super().ping()

# Demonstration of Cooperative Super Execution
print("--- Invoking d.ping() with Cooperative Super ---")
d = D()
d.ping()

print("\n--- Formal Method Resolution Order (MRO) of Class D ---")
for idx, cls in enumerate(D.mro(), start=1):
print(f" Step {idx}: {cls.__name__}")
```

4. Execution Trace Analysis:
1. d.ping() enters D.ping() -> Prints "Ping from D".
2. D calls super().ping() -> Lookups next in MRO: Class B.
3. B.ping() prints "Ping from B" -> calls super().ping().
4. Next in MRO after B is Class C (NOT class A!) -> Prints "Ping from C".
5. C.ping() calls super().ping() -> Next in MRO is Class A -> Prints "Ping from A".
6. A.ping() terminates cleanly.

5. Complexity Analysis:
- MRO Calculation Time: O(V + E) linear graph linearization at class creation time.
- Runtime Dispatch Time: O(1) index array access.

>> Solution to Question 8: Explain Polymorphism and Duck Typing in Python. Contrast Nominal Subtyping (isinstance) with Structural Subtyping (typing.Protocol).

implement the expected interface.
- Nominal Typing (Java/C++): An object is valid only if it explicitly inherits from a declared interface or base class.
- Python Duck Typing: 'If it walks like a duck and quacks like a duck, it's a duck.' Python inspects whether an object possesses the required methods at runtime, regardless of its inheritance pedigree!

2. Modern Formalization: Structural Subtyping via typing.Protocol (PEP 544):
Introduced in Python 3.8, Protocol allows defining static interfaces that check duck-typed structures during mypy analysis without requiring concrete inheritance at runtime.

3. Python Implementation:

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class AudioRenderer(Protocol):
def render_audio(self, stream_data: bytes) -> str:
"""Interface protocol."""
...

# Concrete classes sharing NO common base class
class SpeakerSystem:
def render_audio(self, stream_data: bytes) -> str:
return f"Outputting {len(stream_data)} bytes to High-Fidelity Speakers"

class BluetoothHeadset:
def render_audio(self, stream_data: bytes) -> str:
return f"Streaming {len(stream_data)} bytes via Bluetooth A2DP codec"

class MockAudioSink:
def render_audio(self, stream_data: bytes) -> str:
return f"DevNull Sink swallowed {len(stream_data)} bytes"

# Polymorphic dispatcher function
def broadcast_sound(sink: AudioRenderer, raw_audio: bytes):
# Works seamlessly with ANY duck-typed object implementing render_audio!
print(sink.render_audio(raw_audio))

# Demonstration
payload = b"PCM_AUDIO_DATA_SAMPLES_16BIT"
broadcast_sound(SpeakerSystem(), payload)
broadcast_sound(BluetoothHeadset(), payload)
broadcast_sound(MockAudioSink(), payload)

print(f"\nIs SpeakerSystem an instance of AudioRenderer Protocol? {isinstance(SpeakerSystem(),
AudioRenderer)}")
```

4. Complexity Analysis:
- Dispatch Time: O(1) dynamic method table lookup.
- Space Complexity: O(1) zero inheritance overhead.

>> Solution to Question 9:

[Problem Statement]: Explain Operator Overloading in Python. Detail Arithmetic (__add__, __mul__), Relational (__eq__, __lt__), and String Rep
[Detailed Step-by-Step Answer]:
1. Theoretical Basis of Operator Overloading:
Operator Overloading enables user-defined objects to interact with standard Python operators
(+, -, *, ==, <, etc.) using idiomatic mathematical syntax:
When Python evaluates a + b:
1. Calls type(a).__add__(a, b).
2. If NotImplemented is returned, calls type(b).__radd__(b, a).
3. If still unsupported, raises TypeError.

2. Distinction Between __str__ and __repr__:
- __repr__(self): Unambiguous representation primarily for developers and debugging. Should
ideally look like valid Python code to recreate the object (e.g. ComplexNumber(3, 4)).
- __str__(self): Informal, readable representation formatted for end users (e.g. 3 + 4i).

3. Python Implementation (Complex Number Class):

```
class ComplexNumber:
def __init__(self, real: float, imag: float):
self.real = float(real)
self.imag = float(imag)

def __repr__(self) -> str:
return f"ComplexNumber({self.real}, {self.imag})"

def __str__(self) -> str:
op = "+" if self.imag >= 0 else "-"
return f"({self.real} {op} {abs(self.imag)}j)"

def __add__(self, other) -> 'ComplexNumber':
if isinstance(other, (int, float)):
return ComplexNumber(self.real + other, self.imag)
if isinstance(other, ComplexNumber):
return ComplexNumber(self.real + other.real, self.imag + other.imag)
return NotImplemented

def __radd__(self, other) -> 'ComplexNumber':
return self.__add__(other)

def __mul__(self, other) -> 'ComplexNumber':
# (a + bj)*(c + dj) = (ac - bd) + (ad + bc)j
if isinstance(other, ComplexNumber):
r = (self.real * other.real) - (self.imag * other.imag)
i = (self.real * other.imag) + (self.imag * other.real)
return ComplexNumber(r, i)
return NotImplemented

def __eq__(self, other) -> bool:
if isinstance(other, ComplexNumber):
return self.real == other.real and self.imag == other.imag
return False

# Demonstration
z1 = ComplexNumber(3, 4)
z2 = ComplexNumber(1, -2)
```

```
print(f"z1 via str(): {z1} | via repr(): {repr(z1)}")
print(f"z2 via str(): {z2} | via repr(): {repr(z2)}")
print(f"z1 + z2 = {z1 + z2}")
print(f"z1 + 10 = {z1 + 10} (Commutative right add: {10 + z1})")
print(f"z1 * z2 = {z1 * z2}")
print(f"z1 == z2: {z1 == z2}")
```

4. Complexity Analysis:
- Time Complexity: O(1) arithmetic operations.
- Space Complexity: O(1) new instance creation.

>>P Solution to Question 10: Explain the Container Protocol and Sequence Emulation in Python: __len__, __getitem__, __setitem__, __delitem__, __contains__, an

```
  def __repr__(self) -> str:
  return f"CircularBuffer({self._items}, cap={self._cap})"

  # Demonstration
  cb = CircularBuffer(3)
  cb.append("A")
  cb.append("B")
  cb.append("C")
  print("Buffer Full:", cb)

  cb.append("D") # Evicts 'A'
  print("After appending 'D':", cb)

  print(f"len(cb): {len(cb)}")
  print(f"cb[0] Indexing: {cb[0]}")
  print(f"'B' in cb: {'B' in cb}")
  print(f"'A' in cb: {'A' in cb}")

  # Sequence Iteration in for-loop
  print("Iterating over custom buffer:")
  for item in cb:
  print(f" Item: {item}")

  3. Complexity Analysis:
  - Indexing / Length: O(1) time.
  - Membership / Iteration: O(N) time.
  - Space Complexity: O(capacity) bounded memory.
```

>>> Solution to Question 11 Explain the slots Optimization in Python. Analyze how eliminating instance dict reduces memory footprint and prevents dynar

```
def __init__(self, x: float, y: float):
self.x = x
self.y = y

class SlottedPoint:
__slots__ = ('x', 'y') # Eliminate __dict__

def __init__(self, x: float, y: float):
self.x = x
self.y = y

p_std = StandardPoint(10.0, 20.0)
p_slot = SlottedPoint(10.0, 20.0)

# Memory Footprint Comparison
std_size = sys.getsizeof(p_std) + sys.getsizeof(p_std.__dict__)
slot_size = sys.getsizeof(p_slot)

print(f"Standard Object + __dict__ Size: {std_size} bytes")
print(f"Slotted Object Size: {slot_size} bytes")
print(f"Memory Reduction: {(std_size - slot_size) / std_size * 100:.1f}% savings!")

# Prohibiting undeclared attributes
try:
p_slot.z = 30.0
except AttributeError as ex:
print(f"Caught Expected AttributeError: {ex}")

4. Complexity Analysis:
- Attribute Access Time: O(1) direct struct pointer offset.
- Space Complexity: O(1) minimal fixed C-struct memory.
```

>> Solution to Question 12: Explain Abstract Base Classes (ABCs) in Python. Detail the abc module, @abstractmethod decorator, interface enforcement, and virtual subclasses.

```
@abstractmethod
def process_payment(self, amount: float, currency: str) -> dict:
"""Must be overridden by concrete payment processors."""
pass

@abstractmethod
def refund_payment(self, transaction_id: str) -> bool:
pass

def log_transaction(self, tx_id: str, status: str):
"""Concrete helper method shared by all subclasses."""
print(f"[AUDIT LOG] TX: {tx_id} | Status: {status}")

class StripeProcessor(PaymentGateway):
def process_payment(self, amount: float, currency: str) -> dict:
tx = f"stripe_tx_{int(amount*100)}"
self.log_transaction(tx, "SUCCESS")
return {"id": tx, "status": "approved", "amount": amount}

def refund_payment(self, transaction_id: str) -> bool:
self.log_transaction(transaction_id, "REFUNDED")
return True

# Attempting direct instantiation of ABC raises TypeError
try:
PaymentGateway()
except TypeError as ex:
print(f"Caught ABC Instantiation Error: {ex}")

# Verification of Concrete Processor
gateway = StripeProcessor()
tx_res = gateway.process_payment(250.0, "USD")
print("Payment Processed:", tx_res)

4. Complexity Analysis:
- Interface Check Overhead: O(1) at instantiation time.
- Space Complexity: O(1) class inheritance tables.
```

>>Solution to Question 13: Implement a Banking Domain Model in Python using OOP: BankAccount, SavingsAccount (with interest), and CheckingAccount (with o

```
def __init__(self, account_num: str, owner: str, balance=0.0):
self.account_num = account_num
self.owner = owner
self._balance = float(balance)

@property
def balance(self) -> float:
return self._balance

def deposit(self, amount: float):
if amount <= 0:
raise ValueError("Deposit amount must be positive.")
self._balance += amount
print(f"[{self.account_num}] Deposited ${amount:,.2f}. Balance: ${self._balance:,.2f}")

def withdraw(self, amount: float):
if amount <= 0:
raise ValueError("Withdrawal amount must be positive.")
if amount > self._balance:
raise ValueError(f"Insufficient funds. Balance: ${self._balance:,.2f}")
self._balance -= amount
print(f"[{self.account_num}] Withdrawn ${amount:,.2f}. Balance: ${self._balance:,.2f}")

class SavingsAccount(BankAccount):
def __init__(self, account_num: str, owner: str, balance=0.0, interest_rate=0.04):
super().__init__(account_num, owner, balance)
self.interest_rate = interest_rate

def apply_interest(self):
earned = self._balance * self.interest_rate
self._balance += earned
print(f"[{self.account_num}] Applied ${earned:,.2f} interest. Balance: ${self._balance:,.2f}")

class CheckingAccount(BankAccount):
def __init__(self, account_num: str, owner: str, balance=0.0, overdraft_limit=1000.0):
super().__init__(account_num, owner, balance)
self.overdraft_limit = overdraft_limit

def withdraw(self, amount: float):
"""Overridden withdraw supporting overdraft buffer."""
if amount <= 0:
raise ValueError("Withdrawal amount must be positive.")
if amount > (self._balance + self.overdraft_limit):
raise ValueError("Transaction declined: Exceeds maximum overdraft limit.")
self._balance -= amount
print(f"[{self.account_num}] Overdraft Withdrawn ${amount:,.2f}. Balance:
${self._balance:,.2f}")

# Verification
sav = SavingsAccount("SA-101", "Aarav", balance=10_000.0, interest_rate=0.05)
sav.apply_interest()

chk = CheckingAccount("CA-202", "Priya", balance=500.0, overdraft_limit=500.0)
chk.withdraw(800.0) # Enters overdraft of -$300
```

3. Complexity Analysis:
- Time Complexity: O(1) for deposit, withdraw, and interest calculations.
- Space Complexity: O(1) memory per account instance.

Solution to Question 14: Implement the Singleton Design Pattern in Python using both the `__new__` override technique and a custom Metaclass. Analyze thread

```
print(f"pool1 is pool2: {pool1 is pool2} (Same Object ID: {id(pool1)})")

cfg1 = AppConfiguration()
cfg2 = AppConfiguration()
cfg1.theme = "Light"
print(f"cfg1 is cfg2: {cfg1 is cfg2} | cfg2.theme: {cfg2.theme}")

4. Complexity Analysis:
- Lookup Time: O(1) dictionary key lookup.
- Space Complexity: O(1) singleton memory instance.
```

>>P Solution to Question 15: Implement a Custom Matrix Class in Python with complete Operator Overloading (+, -, *, ==, str, repr, transpose) and dimension validat

```
  trans = [[self.data[r][c] for r in range(self.rows)] for c in range(self.cols)]
  return Matrix(trans)

  def __eq__(self, other) -> bool:
  return isinstance(other, Matrix) and self.data == other.data

  # Demonstration
  m1 = Matrix([[1, 2], [3, 4]])
  m2 = Matrix([[5, 6], [7, 8]])

  print("Matrix 1:\n", m1)
  print("Matrix 2:\n", m2)
  print("m1 + m2 =\n", m1 + m2)
  print("m1 * m2 =\n", m1 * m2)
  print("Transpose of m1:\n", m1.transpose())
```

3. Complexity Analysis:
- Addition Time: O(R * C).
- Multiplication Time: O(R1 * C1 * C2).
- Space Complexity: O(R * C) memory for new Matrix instance.

>>P Solution to Question 16 Explain the Descriptor Protocol in Python (`__get__`, `__set__`, `__delete__`). Implement a TypeChecked and BoundedNumeric descriptor

```
raise ValueError(f"Value {value} out of bounds [{self.min_val}, {self.max_val}].")
setattr(instance, self.storage_name, float(value))

class ChemistryExperiment:
# Descriptors managing class attributes
ph_level = BoundedFloat(0.0, 14.0)
temperature_c = BoundedFloat(-273.15, 1000.0)

def __init__(self, ph: float, temp: float):
self.ph_level = ph
self.temperature_c = temp

# Demonstration
exp = ChemistryExperiment(7.2, 25.0)
print(f"Experiment pH: {exp.ph_level} | Temp: {exp.temperature_c} C")

# Validation Failure Test
try:
exp.ph_level = 18.5 # Exceeds max pH 14.0
except ValueError as err:
print(f"Descriptor Validation Caught: {err}")
```

3. Complexity Analysis:
- Lookup & Validation Time: O(1) attribute access.
- Space Complexity: O(1) descriptor metadata on class object.

>> Solution to Question 17: Explain Mixin Classes and Cooperative Multiple Inheritance in Python. Implement a JSONSerializable and LoggingMixin system.

```
  self.sku = sku
  self.title = title
  self.price = price
  self.log_event("INITIALIZATION")

  def update_price(self, new_price: float):
  self.price = new_price
  self.log_event(f"PRICE_UPDATE -> ${new_price:.2f}")

  # Demonstration
  prod = ProductCatalogItem("SKU-9801", "Mechanical Keyboard", 129.99)
  prod.update_price(119.99)
  print("\nExported JSON via Mixin:")
  print(prod.to_json())

  3. Complexity Analysis:
  - Time Complexity: O(K) where K is number of instance fields during JSON serialization.
  - Space Complexity: O(1) shared mixin methods.
```

>> Solution to Question 18: Implement a State Machine Pattern in Python using OOP: OrderState, PendingState, PaidState, ShippedState, and DeliveredState.

```
def deliver(self, ctx: OrderContext): pass

class PendingState(OrderState):
def pay(self, ctx: OrderContext): ctx.set_state(PaidState())
def ship(self, ctx: OrderContext): print("Cannot ship unpaid order!")
def deliver(self, ctx: OrderContext): print("Cannot deliver pending order!")

class PaidState(OrderState):
def pay(self, ctx: OrderContext): print("Order is already paid.")
def ship(self, ctx: OrderContext): ctx.set_state(ShippedState())
def deliver(self, ctx: OrderContext): print("Cannot deliver unshipped order.")

class ShippedState(OrderState):
def pay(self, ctx: OrderContext): print("Already paid.")
def ship(self, ctx: OrderContext): print("Already shipped.")
def deliver(self, ctx: OrderContext): ctx.set_state(DeliveredState())

class DeliveredState(OrderState):
def pay(self, ctx): print("Order completed.")
def ship(self, ctx): print("Order completed.")
def deliver(self, ctx): print("Already delivered.")

# Verification
order = OrderContext("ORD-9001")
order.ship() # Rejected
order.pay() # Transition to Paid
order.ship() # Transition to Shipped
order.deliver() # Transition to Delivered

3. Complexity Analysis:
- State Transition Time: O(1) constant time.
- Space Complexity: O(1) state instance references.
```

>> Solution to Question 19: Explain the @dataclass Decorator (PEP 557) in Python. Contrast dataclasses with standard classes, namedtuples, and pydantic mode

```
@dataclass(order=True, frozen=True)
class SensorReading:
# Sort index used for ordering comparisons
sort_index: float = field(init=False, repr=False)
timestamp: float
sensor_id: str
value: float

def __post_init__(self):
# Calculate sort_index based on value for ordering
object.__setattr__(self, 'sort_index', self.value)

# Demonstration
r1 = SensorReading(timestamp=1700000.0, sensor_id="TEMP-01", value=24.5)
r2 = SensorReading(timestamp=1700010.0, sensor_id="TEMP-02", value=31.2)

print(f"Dataclass Auto __repr__: {r1}")
print(f"Dataclass Auto __eq__: {r1 == r2}")
print(f"Dataclass Auto __lt__: {r1 < r2} (24.5 < 31.2)")

# Frozen immutability check
try:
r1.value = 99.0
except Exception as ex:
print(f"Caught FrozenInstanceError: {type(ex).__name__}")

4. Complexity Analysis:
- Instantiation Time: O(K) where K is number of fields.
- Space Complexity: O(K) memory per instance.
```

>> Solution to Question 20: Implement a Custom Iterator and Iterable Class in Python: FibonacciIterable supporting the Iterator Protocol (__iter__ and __next__).

```
self.limit = limit
self.count = 0
self.a = 0
self.b = 1

def __iter__(self):
return self

def __next__(self) -> int:
if self.count >= self.limit:
raise StopIteration

val = self.a
self.a, self.b = self.b, self.a + self.b
self.count += 1
return val

# Demonstration
fib_seq = FibonacciIterable(7)

print("First Iteration:")
for num in fib_seq:
print(num, end=" ")
print()

print("Second Independent Iteration (Fresh Iterator Created):")
print(list(fib_seq))
```

3. Complexity Analysis:
- Time Complexity: O(1) per __next__() step.
- Space Complexity: O(1) auxiliary state.

>>P Solution Statement Question 21: Implement a Dependency Injection Container in Python using Classes and Reflection for service registration and resolution.

```
class IEmailService:
def send(self, to, msg): pass

class SMTPEmailService(IEmailService):
def send(self, to, msg):
print(f"Sending SMTP email to {to}: '{msg}'")

class NotificationManager:
def __init__(self, email_service: IEmailService):
self.email_svc = email_service

def notify(self, user, message):
self.email_svc.send(user, message)

# Assembly via DI Container
container = DIContainer()
container.register_singleton(IEmailService, SMTPEmailService())

email_svc = container.resolve(IEmailService)
notifier = NotificationManager(email_svc)
notifier.notify("user@cse.edu", "Exam Schedule Released")
```

3. Complexity Analysis:
- Registration and Resolution Time: O(1) dictionary access.
- Space Complexity: O(S) services registered.

>> Solution to Question 22: Implement the Observer Design Pattern in Python using Classes: EventDispatcher, Subject, and Concrete Observers.

```
  self._observers = []

  def subscribe(self, observer: IObserver):
  self._observers.append(observer)

  def notify_all(self, event_type: str, data: dict):
  for obs in self._observers:
  obs.on_event(event_type, data)

  def login(self, username: str, password_attempt: str):
  if password_attempt == "correct":
  self.notify_all("LOGIN_SUCCESS", {"user": username})
  else:
  self.notify_all("LOGIN_FAILED", {"user": username, "ip": "10.0.0.1"})

  # Demonstration
  auth = AuthenticationService()
  auth.subscribe(SecurityAlertSystem())
  auth.subscribe(AuditLogSystem())

  auth.login("admin", "wrong_pass")

  3. Complexity Analysis:
  - Notification Time: O(N) where N is number of observers.
  - Space Complexity: O(N) observer list.
```

Solution to Question 23: >>> Implement a Custom Context Manager in Python using OOP: DatabaseConnectionManager implementing enter and exit with

```
  print(f"[TX COMMIT] Transaction committed successfully.")
  return True

  # Verification
  with DatabaseTransaction("Production_DB") as tx:
  tx.query("UPDATE accounts SET balance = balance - 100 WHERE id = 1;")
  tx.query("UPDATE accounts SET balance = balance + 100 WHERE id = 2;")
```

3. Complexity Analysis:
- Overhead: O(1) __enter__ and __exit__ lifecycle calls.

Question 24: Implement an Object Relational Mapping (ORM) Style Active Record Entity in Python using Metaclasses and Field descriptors.

Solution:

```
print(f"Caught Expected Type Validation Error: {ex}")
```

3. Complexity Analysis:
- Validation Time: O(1) type check.
- Space Complexity: O(1) descriptor instances on class.

>>Solution Question 28: Custom Descriptor Hierarchy for Python Classes: Payment Platform with Card Validation and Crypto

[Detailed Step-by-Step Answer]:
1. Theoretical Principles of Exception Hierarchies:
User-defined exceptions should inherit from Python's built-in Exception class (never BaseException).
Structuring exceptions in a domain hierarchy allows callers to catch broad categories of errors or handle fine-grained failures individually.

2. Python Implementation:

```
class ECommerceError(Exception):
"""Base exception for all domain errors."""
pass

class InventoryError(ECommerceError):
"""Errors related to stock availability."""
pass

class OutOfStockError(InventoryError):
def __init__(self, sku: str, requested: int, available: int):
super().__init__(f"SKU {sku} is out of stock. Requested: {requested}, Available: {available}")
self.sku = sku

class PaymentError(ECommerceError):
"""Errors related to transaction processing."""
pass

# Demonstration
def order_product(sku: str, qty: int, stock: int):
if qty > stock:
raise OutOfStockError(sku, qty, stock)
print(f"Order placed for {qty} units of {sku}")

try:
order_product("SKU-100", 5, 2)
except OutOfStockError as err:
print(f"Caught Specific OutOfStockError: {err}")
except ECommerceError as err:
print(f"Caught Broad Domain Error: {err}")
```

3. Complexity Analysis:
- Exception Instantiation Time: O(1).
- Space Complexity: O(Depth) traceback frames.

>> Solution to Question 27: Implement the Factory Method and Abstract Factory Design Patterns in Python using Classes for Cross-Platform UI Rendering.

```
from abc import ABC, abstractmethod

class IButton(ABC):
@abstractmethod
def render(self) -> str: pass

class WindowsButton(IButton):
def render(self): return "[Windows Fluent Button]"

class MacOSButton(IButton):
def render(self): return "(macOS Aqua Button)"

class IUIFactory(ABC):
@abstractmethod
def create_button(self) -> IButton: pass

class WindowsUIFactory(IUIFactory):
def create_button(self): return WindowsButton()

class MacOSUIFactory(IUIFactory):
def create_button(self): return MacOSButton()

# Client Code
def build_ui(factory: IUIFactory):
btn = factory.create_button()
print("Rendered UI Element:", btn.render())

build_ui(WindowsUIFactory())
build_ui(MacOSUIFactory())
```

3. Complexity Analysis:
- Factory Creation Time: O(1).
- Space Complexity: O(1) UI component instances.

>> Solution to Question 28: Explain Metaclasses in Python in depth. Implement a ModelValidatorMeta metaclass that validates that all subclass attributes have doc

```
# Valid Class
class SecureService(metaclass=DocumentedClassMeta):
"""This class manages secure communications."""
pass

print("SecureService successfully compiled with docstring validation.")

# Invalid Class
try:
class UndocumentedService(metaclass=DocumentedClassMeta):
pass
except TypeError as err:
print(f"Metaclass Enforcement Caught: {err}")
```

3. Complexity Analysis:
- Metaclass Overhead: Executed once at class definition time; zero runtime call penalty.

>> Solution to Question 29: Implement a Thread-Safe In-Memory Key-Value Store with TTL (Time-To-Live) Eviction in Python using Classes and threading.Lock.

```
print("Immediate Get:", cache.get("session_token"))
time.sleep(1.1)
print("After TTL Expiry Get:", cache.get("session_token"))
```

3. Complexity Analysis:
- Get / Set Time Complexity: O(1) hash table lookup guarded by mutex.
- Space Complexity: O(K) stored items.

>> Solution to Question 30: Implement the Command Design Pattern in Python with Undo/Redo Capability using Classes and Command Stacks.

```
doc = TextDocument()
editor = EditorInvoker()

editor.execute_command(AppendTextCommand(doc, "Hello "))
editor.execute_command(AppendTextCommand(doc, "World!"))
print(f"Current Document: '{doc.text}'")

editor.undo()
print(f"After Undo: '{doc.text}'")
```

3. Complexity Analysis:
- Time Complexity: O(1) command execution and undo stack operations.
- Space Complexity: O(History) command objects.
